

Day 1 – Shell Script Assignment

1. Shell Script

The following shell script performs the requested operations.

Shell Script

```
#!/bin/bash
```

```
# 1. Create a folder named 'sample' in the home directory
mkdir -p "$HOME/sample"
```

```
# Move into the sample directory
cd "$HOME/sample" || exit 1
```

```
# 2. Create sample.txt
touch sample.txt
```

```
# 3. Add content to sample.txt
echo "Hi! This is just a sample text file created using shell script." > sample.txt
```

```
# 4. Print the contents of the file
echo "Contents of sample.txt:"
cat sample.txt
```

```
# 5. Print the number of occurrences of letter 't'
echo "Number of occurrences of letter 't':"
grep -o "t" sample.txt | wc -l
```

```
# 6. Give the owner read, write and execute permissions
chmod u+rxw sample.txt
```

```
# 7. Append another line to sample.txt
echo "Hi! This is just another sample text added to file." >> sample.txt
```

```
# 8. Give the group only read permission
chmod g=r sample.txt
```

```
# 9. Deny all permissions to all users
chmod 000 sample.txt
```

```
# Temporarily restore owner permissions so that subsequent operations can be performed
chmod u+rxw sample.txt
```

```
# 10. Create sample2.txt with content similar to sample.txt
cp sample.txt sample2.txt
```

```
# 11. Add 1000 random lines to sample.txt
for i in {1..1000}
do
    echo "Random line $RANDOM $i" >> sample.txt
done
```

```
# 12. Print the top 50 lines
echo "Top 50 lines:"
head -n 50 sample.txt

# 13. Print the bottom 50 lines
echo "Bottom 50 lines:"
tail -n 50 sample.txt

# 14. Create five files
touch prog1.txt prog2.txt program.txt code.txt info.txt

# 15. List files containing "prog" in their name
echo "Files containing 'prog':"
ls | grep "prog"

# 16. Create an alias so that 'list prog' produces the same output
alias list='ls | grep'

# Final permission setting: deny all access to sample.txt
chmod 000 sample.txt

echo "Script execution completed."
```

2. Difference Between **source** and **sh**

source

source script.sh

source executes the script in the **current shell**.

Any variables, functions, or aliases created by the script remain available in the current shell.

It can also be written as:

. script.sh

sh

sh script.sh

sh executes the script using a **separate shell process**.

Variables created inside the script normally do not remain available in the current shell after the script finishes.

Difference

source

Runs in current shell

sh

Runs in a separate shell

Changes to variables/environment can remain

Changes normally disappear after execution

Useful for loading shell configuration

Used to execute a shell script

3. Difference Between **a.txt** and **b.txt**

Create the files:

```
touch a.txt b.txt
```

To compare their contents:

```
diff a.txt b.txt
```

diff shows the differences between the two files.

Another command is:

```
cmp a.txt b.txt
```

For line-by-line comparison, **diff** is commonly used.

4. Difference Between **ls** and **lsuf**

ls

ls is used to **list files and directories**.

Example:

```
ls
```

For detailed information:

```
ls -l
```

lsuf

lsuf means **List Open Files**.

It is used to find files and resources being used by running processes.

Example:

```
lsuf
```

To check which process is using port 8080:

```
lsuf -i :8080
```

Difference

Command	Use
<code>ls</code>	Lists files and directories
<code>ls -l</code>	Lists files/resources opened by processes

5. Create `./hello/world` Using One Command

Use:

```
mkdir -p ./hello/world
```

The `-p` option creates both directories if they do not already exist.

Result:

```
hello/  
└── world/
```

6. Permanently Set an Environment Variable

For Bash, an environment variable can be added to `~/.bashrc`.

Example:

```
export MY_VARIABLE="Hello"
```

To add it permanently:

```
echo 'export MY_VARIABLE="Hello"' >> ~/.bashrc
```

Reload the file:

```
source ~/.bashrc
```

Check the variable:

```
echo $MY_VARIABLE
```

Output:

```
Hello
```

The variable will be available in future Bash sessions.

7. Find and Kill a Process Running on a Port

Suppose a process is running on port `8080`.

Step 1: Find the process

`lsof -i :8080`

Example:

COMMAND	PID	USER	FD	TYPE	DEVICE	NODE	NAME
java	12345	user	...	TCP	...	...	*:8080

The PID is `12345`.

Step 2: Kill the process

`kill 12345`

If the process does not stop, use:

`kill -9 12345`

We can also directly get the PID using:

`lsof -t -i :8080`

Then:

`kill $(lsof -t -i :8080)`

Conclusion

This assignment demonstrates several important Linux and Bash concepts, including:

- Creating and manipulating files and directories
- Reading and modifying file contents
- File permissions using `chmod`
- File comparison using `diff`
- Process and port management using `lsof`
- Directory creation using `mkdir -p`
- Environment variables
- Bash aliases
- Viewing portions of large files using `head` and `tail`
- Executing scripts using `source` and `sh`